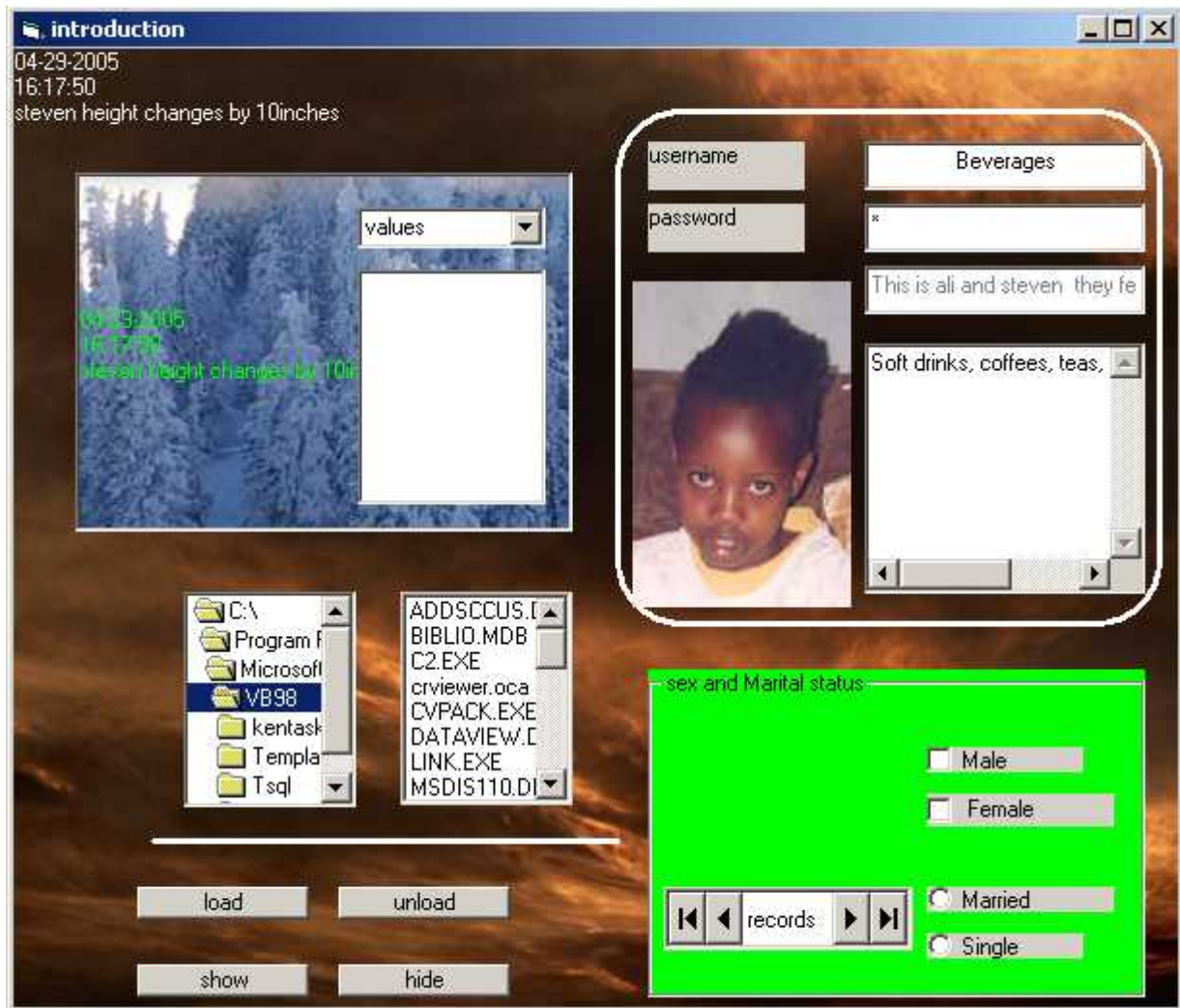


VISUAL BASIC PROGRAMMING

PRACTICAL ASSIGNMENTS

BY: Edwin Eddy Mukenya E_MAIL: empisth@yahoo.com

1. Create the interface below in VISUAL BASIC 6. Label the objects appropriately, and assign the relevant characteristic adjusted from the property window.



1. Highlight the code view.
2. On the **View** menu, select **Code** (or right-click on the interface, then select **Code view**).
3. Within the code editor, type the code below:

```
Private Sub cmdhide_Click()
```

```
MDIForm1.Hide
```

```
End Sub
```

```
Private Sub cmdload_Click()
```

```
Load MDIForm1 'loads aform in memory but does not display
```

```
End Sub
```

```
Private Sub cmdshow_Click()
```

```
MDIForm1.Show 'displays aform on the screen
```

```
End Sub
```

```
Private Sub cmdunload_Click()
Unload MDIForm1      'removes the form from memory and display
End Sub

Private Sub Form_Load()
  Frone.BackColor = vbGreen
End Sub

Private Sub Timer1_Timer()
  Static t As Integer
  frmtwo.Cls
  frmtwo.Print Date$
  frmtwo.Print Time$
  frmtwo.Print "steven height changes by " & t & "inches"

  pictone.Cls
  pictone.CurrentY = 1000

  pictone.Print Date$
  pictone.Print Time$
  pictone.Print "steven height changes by " & t & "inches"
  t = t + 1
End Sub
```

SPLASH SCREEN

1. Create the splash screen below:



2. Then type the code below to support its operation

```
Private Sub Timer1_Timer()  
    Unload Me  
End Sub
```

DECLARATION OF VARIABLES IN VISUAL BASIC

DECLARATION OF ARRAYS

'syntax

'<keyword>{array name}[dimension] as <type>

Dim Data(56) As Long

'<keyword>{arrayname}[lower limit to upper limit] as <type>

Dim data1(2 To 56) As String

'<keyword>{arrayname}() as <type>

Dim toll() As Double

'constant value

'syntax

'const <varname> = value

Const pie = 3.142

'EXPLICIT DECLARATION

'syntax

'<keyword>{varname} as <type>

Dim sStr As String

Public nNum As Integer

Private vVal As Variant

' Static Ddate As Date

Private Sub Form_Load()

ReDim toll(1 To 30) **Assign size to dynamically declared array**

'fixed length declaration

'syntax

'<keyword>{varname} as string * stringlength

Dim sName As String * 25

'initialization

sStr = ""

nNum = 0

vVal = ""

Ddate = #1/2/2004#

'IMPLICIT DECLARATION

nnum1% = 56

'integer declaration

sSrt2\$ = "steven"

'string declaration

vVal1& = 78

'long declaration

cMoney@ = 789#

'currency declaration

fDec! = 890.78

'single declaration

dVal# = 567.32

'double declaration

Design the interface below then type the proceeding code in the code view

Assignment 1.0 (arrays)

Returns true if the variable data is an array otherwise false.

```
Private Sub Label1_Click()  
    Text1.Text = IsArray(Data)  
End Sub
```

Returns the data type of the array

```
Private Sub Label2_Click()  
    Text2.Text = TypeName(Data)  
End Sub
```

Upper limit of the array data

```
Private Sub Label3_Click()  
    Text3.Text = UBound(Data)  
    Debug.Print UBound(Data) 'printing the upper bound of the array in to the debug window
```

End Sub

Lower limit of the array data

```
Private Sub Label4_Click()
    Text4.Text = LBound(Data)
End Sub
```

Length of the array named data

```
Private Sub Label5_Click()
    Text5.Text = UBound(Data) - LBound(Data) + 1
End Sub
```

Assignment 1.1(assigning values into arrays)

Initializing array data

```
'data(1) = 0

For k% = 0 To UBound(Data)
    Data(k) = 0
Next k
```

'Assigning values into an array

```
Data(0) = 12
Data(1) = 56      'Static or fixed values
Data(2) = 78

For j% = 0 To UBound(Data)
    Data(j) = CLng(Rnd * 200) 'Randomly assigned values
Next j
```

End Sub

Assignment 1.2(retrieving values from arrays)

Displaying the data in the list box

```
Private Sub Command1_Click()
    On Error GoTo kitoto
    For j% = 0 To UBound(Data)
        List1.AddItem Data(j)
    Next j

kitoto:
    MsgBox Err.Source & Space(3) & Err.Description & Space(3) & Err.Number

End Sub
```

CONTROL STRUCTURES IN VISUAL BASIC

Introduction

'Data Types

'Variant supports all the other variables

'integer.....fixed or whole numbers

'long

'Byte

'doublefloating point numbers

'float

'string.....text or sequence of characters

'date.....calendar

'boolean.....logic{true or false}

'Declaring Variable

'should not be a number

'should not be alphanumeric

'should not be keywords or symbols

'should not have spaces

'should not include math functions

'Syntax

'<keyword>[varname] as <datatype>.....explicitly

'[varname]<special character>=value.....implicitly

Implement the form interface below and type the following code in the form design view and ensure that it meets the object procedure requirements

Form4

WhileDo

ForToDo

ForEach

DoUntil

DoWhile

DoLoopWhile

SelectCase

IfThen

IfThenElse

Label1

Dim vVal As Variant, vNum As Variant (**Declaring variant variables**)

```
Private Sub Command1_Click()
```

```
  'syntax
```

```
  'while <condition is true>
```

```
    'statements
```

```
  'wend
```

```
  Form4.Cls
```

```
  Dim sK As String
```

```
  List1.Clear
```

```
  While sK <> "N"
```

```
    List1.AddItem sK
```

```
    sK = InputBox("Enter Names", "WHILE")
```

```
  Wend
```

```
End Sub
```

```
Private Sub Command2_Click()
```

```
  'syntax
```

```
  'for variablename=initial value to max value
```

```
    'do statements
```

```
  'next
```

```
  List1.Clear
```

```
  Dim A(20) As Integer
```

```
  For i% = 0 To 20 Step (1)
```

```
    A(i) = CInt(Rnd * 20)
```

```
    List1.AddItem A(i)
```

```
  Next i
```

```
End Sub
```

```
Private Sub Command3_Click()
```

```
  'Syntax
```

```
  'for each (varname) in vararray
```

```
    'statement
```

```
  'next
```

```
  List1.Clear
```

```
  For Each vNum In vVal
```

```
    List1.AddItem vNum      '...extracting arguments from vval assigning then to
```

```
    '.....vnum one after the other then allocating the textbox
```

```
  Next
```

```
End Sub
```

```
Private Sub Command4_Click()
```

```
  'Syntax
```

```
  'DO UNTIL <CONDITION=false> - loops only and only if the condition is not fulfilled(false)
```

```
  ' STATEMENT
```

```
  'LOOP
```



```

Dim n As Integer '.....declaring variable n as an integer
    n = 0      '.....initializing n to zero
List1.Clear
Do Until n = 100
    List1.AddItem n
    n = n + 1  '.....incrementing n by 1
Loop
End Sub

```

```
Private Sub Command5_Click()
```

'Syntax

```

'do while <condition=true> - loops iff the condition is fulfilled(true)
'    statement
'loop

```

```

List1.Clear
Static l As Long
    l = 100
    Do While l > 0
        List1.AddItem l
        l = l - 1 '....decrementing l by 1
    Loop
End Sub

```

```
Private Sub Command6_Click()
```

'Syntax

```

'do
'    statement
'loop while <condition=true>

```

```

List1.Clear
Static d As Double
    d = 0
Do
    List1.AddItem d
    d = d + 1
Loop While d < 100

End Sub

```

```
Private Sub Command7_Click()
```

'Syntax

```

'select case condition expression
'    Case condi
'        statement
'    "
'    "
'    "
'end select

```

```
Dim v As Variant
```

```
v = InputBox("Enter values", "VALUES") 'using input box to enter values
```

```
If IsNumeric(v) = True Then
```

```
    v = Val(v)
```

```
Else
```

```
    v = CStr(v)
```

```
End If
```

```
Select Case v
```

```
    Case 0
```

```
        Label1.Caption = "IT is a zero/indian number"
```

```
    Case 1 To 10000
```

```
        Label1.Caption = "This is a numeric number"
```

```
    Case "a" To "z"
```

```
        Label1.Caption = "This is a lower case alphabet"
```

```
    Case "EDWIN EDDY MUKENYA"
```

```
        Label1.Caption = "I will never die so long as God allows who make me live, eddy"
```

```
    Case Else
```

```
        Label1.Caption = "go to hell"
```

```
End Select
```

```
End Sub
```

```
Private Sub Command8_Click()
```

```
    'Syntax
```

```
    'if <condition = true >
```

```
    'then statement
```

```
    'end if
```

```
End Sub
```

```
Private Sub Command9_Click()
```

```
    'Syntax
```

```
    'if <condition>
```

```
    'then statement
```

```
    'else statement
```

```
    'end if
```

```
End Sub
```

```
Private Sub Form_Load()
```

```
    vVal = Array(1, 2, 3, 4, 5, 6, 7, 8, 9, 0, "sisay", "sleep", "in", "class") 'assigning arguments to a variant variable
```

```
End Sub
```

STRING FUNCTION AND USE

Math function	Use
Abs(Number)	Returns the absolute value of a number
Atn(Number As Double)	Returns the arctangent of a number
Cos(Number As Double)	Returns the cosine of an angle
Exp(Number As Double)	Returns e (the base of natural logarithms) raised to a power
Log(Number As Double)	Returns the natural logarithm of a number
Randomize([Number])	Initializes the random-number generator
Rnd([Number])	Returns a random number
Round(Number, [NumDigitsAfterDecimal As Long])	Round to a given no. of decimal places
Sgn(Number)	Returns an integer indicating the sign of a number
Sin(Number As Double)	Returns the sine of an angle
Sqr(Number As Double)	Returns the square root of a number
Tan(Number As Double)	Returns the tangent of an angle

Conversion built in functions

CCur(Expression) Converts an expression to a Currency
 CDate(Expression) Converts an expression to a Date
 CDbI(Expression) Converts an expression to a Double
 CDec(Expression) Returns a variant with a value converted to a decimal subtype
 CInt(Expression) Converts an expression to an Integer
 CLng(Expression) Converts an expression to a Long
 CSng(Expression) Converts an expression to a Single
 CStr(Expression) Converts an expression to a String
 CVar(Expression) Converts an expression to a Variant
 CVDate(Expression) Converts an expression to a Date
 CVerErr(Expression) Returns a Variant containing a user-specified error number
 Error([ErrorNumber]) Returns the error message for a given error number
 Fix(Number) Returns the integer portion of a number
 Hex(Number) Returns a string representing the hexadecimal value of a number
 Int(Number) Returns the integer portion of a number
 Oct(Number) Returns a string representing the octal value of a number
 Str(Number) Returns a string representation of a number
 Val(String As String) Returns the numbers contained in a string

Inbuilt string functions

Design the interface below and type the code that follows the it; in the corresponding code view.

Convert the code below to fit your form design

```
Private Sub cmbone_Click()
```

```
    'Syntax
```

```
    'val(string)---- returns values
```

```
    txtval.Text = Val(cmbone.Text)
```

```
End Sub
```

```
Private Sub Form_Load()
```

```
    Dim i As Integer          'declaring variable i as an integer counter
```

```
    i = 1                    'initializing counter
```

```
    While i <= 100           'while loop condition statement
```

```
        Combo1.AddItem i     'assigns combo with i items
```

```
        Combo2.AddItem i
```

```
        i = i + 1            'increments the counter
```

```
    Wend                     'end of the loop
```

```
End Sub
```

```
Private Sub Label1_Click()
```

```
    'Syntax
```

```
    'instr([startingpoint,sourcestring,search string,casemode)
```

```
    txtdisplay7.Text = InStr(Val(txtstrtpnt2.Text), txtinput.Text, txtsearch.Text,  
    Val(txtsearch.Text))
```

End Sub

Private Sub Label4_Click()

Dim sSisay As String

sSisay = txtinput.Text

Mid\$(sSisay, Val(Combo2.Text), Val(txtnumchar4.Text)) = txtinput1.Text

txtdisplay3.Text = sSisay

End Sub

Private Sub lblcase_Click()

'Syntax

'lcase(string)----changes the text to lower case (small letters!)

txtlcase.Text = LCase(txtinput.Text)

End Sub

Private Sub lblleft_Click()

'Syntax

'left(sourcestring,numchar)----Extracts specified number of characters from the left part of the string

txtdisplay.Text = Left\$(txtinput.Text, Val(txtnumchar1.Text))

Print Val(txtnumchar1.Text)

End Sub

Private Sub lblllen_Click()

'Syntax

'len(string)-----returns the number of characters typed including the spaces!

txtlen.Text = Len(txtinput.Text)

End Sub

Private Sub lbltrim_Click()

'Syntax

'ltrim(string)----removes spaces from the left part of the string

txtltrim.Text = LTrim(txtinput.Text)

End Sub

Private Sub lblmid_Click()

'Syntax

'mid(sourcestring,[starting point,number of characters)

txtmid.Text = Mid\$(txtinput.Text, Val(Combo1.Text), Val(txtnumchar3.Text))

End Sub

Private Sub lblright_Click()

'Syntax

'right(sourcestring,numchar)----Extracts specified number of characters from the right part of the string

txtdisplay2.Text = Right\$(txtinput.Text, Val(txtnumchar2.Text))

Print Val(txtnumchar2.Text)

End Sub

```
Private Sub lbltrim_Click()
```

```
  'Syntax
```

```
  'rtrim(string)----removes spaces from the right part of the string
```

```
  txttrim.Text = RTrim(txtinput.Text)
```

```
End Sub
```

```
Private Sub lblstrreverse_Click()
```

```
  'Syntax
```

```
  'strreverse(string)-----reverses the given letters from right to left
```

```
  txtstrreverse.Text = StrReverse(txtinput.Text)
```

```
End Sub
```

```
Private Sub LBLTRIM_Click()
```

```
  'Syntax
```

```
  'trim(string)----it removes spaces from the whole string(at the heading and tail)
```

```
  txttrim.Text = Trim(txtinput.Text)
```

```
End Sub
```

```
Private Sub lblucase_Click()
```

```
  'Syntax
```

```
  'ucase(string)-----changes the text to upper case (capitol letters!)
```

```
  txtucase.Text = UCase(txtinput.Text)
```

```
End Sub
```

CREATING AND USING PROCEDURES AND FUNCTIONS IN VB6

To create a module or module class

- i.) **select the menu project**
- ii.) **highlight add module or class module**
- iii.) **in the module or class module dialogue box click on ok and this will give you the module or class module code editor**(they don't support objects and both can be created in Activex .EXE Application and Activex .DLL application)
- iv.) **You can declare public variables, public procedures/subroutines and functions**
- v.) **Modules are saved with extensions .bas and class modules as .class**
- vi.) **Using subroutines from modules in the main program**

```
Private Sub Form_Load()
    oprflag = False
    fillbuttons Command1
    disable frmcalculator
    clearing frmcalculator
End Sub
```

- vii.) **Using routines from class modules in the main program**

```
Dim ali(50) As Long
```

```
Dim jclass As Class1
```

```
Private Sub Command1_Click()
    Set jclass = New Class1
    jclass.selectsort ali

    For i% = 0 To UBound(ali) - 1
        Debug.Print "At Index " & Space(7) & ali(i)
    Next i
End Sub
```

```
Private Sub Form_Load()
    On Error GoTo ismail
    Me.Caption = "debug"

    For i% = 0 To UBound(ali) - 1
        ali(i) = CLng(Rnd * 200)
        Debug.Print "At Index " & Space(7) & ali(i)
    Next i
    Debug.Print "_____ "
ismail:
    Me.Print Err.Description
End Sub
```


viii.) Produres**Syntax**

```
[public,private] sub {procedurename}([arguments])
    statements
end sub
```

ix.) functions

```
[public,private][static] function functionname([arguments]) [as type]
    statement
end function
```

Some procedures and functions created in modules and class modules**PROCEDURES**

```
Public Const pie = 3.142
Public nft As Integer
Public vVal As Variant
```

Filling calculator interface button caption with values

```
Sub fillbuttons(obuttons As Object)
    Dim vNum As Variant, i As Integer
    vVal = Array(1, 2, 3, 4, 5, 6, 7, 8, 9, 0, ".", "clr", "*", "+", "-", "/", "^", "%", "=",
"back")
    For Each vNum In vVal
        obuttons(i).Caption = vNum 'assigning caption names to the command buttons
        i = i + 1
    Next
End Sub
```

Enabling controls

```
Sub enable(frm As Form)

    Dim mycontrol As Control
    For Each mycontrol In frm.Controls
        If TypeOf mycontrol Is TextBox Then
            mycontrol.Enabled = True
        End If
    Next
End Sub
```

Disabling controls

```
Sub disable(frm As Form)

    Dim mycontrol As Control
    For Each mycontrol In frm.Controls
        If TypeOf mycontrol Is TextBox Then
            mycontrol.Enabled = False
            mycontrol.Alignment = vbRightJustify
        End If
    Next
End Sub
```

Clearing control textbox

```

Sub clearing(frm As Form)
    Dim mycontrol As Control
    For Each mycontrol In frm.Controls
        If TypeOf mycontrol Is TextBox Then
            mycontrol.Text = ""
        End If
    Next
End Sub

```

OR

```

Sub cleartextbox(frm As Form)
    Dim otext As TextBox
    For Each otext In frm.Text1
        otext.Text = ""
    Next
End Sub

```

FUNCTIONS

```

Public Function binary (lnum As Long)
    Dim lp As Long, sOut As String
    While lnum <> 0
        lp = lnum Mod 2
        lnum = lnum \ 2
        sOut = sOut & lp
    Wend
    binary = StrReverse(sOut)
End Function

Public Function fact(n As Integer)
    If n = 0 Then
        fact = 1
    Else
        fact = n * fact(n - 1)
    End If
End Function

```

```

Public Function fibb(n As Integer)
    If n < 3 Then
        fibb = 1
    Else
        fibb = fibb(n - 1) + fibb(n - 2)
    End If
End Function

```

```

Public Function inverse(n As Integer)
    If n <> 0 Then
        inverse = 1 / n
    Else
        inverse = "0"
    End If
End Function

```

```

End If
End Function

Public Function arith(oprsign As String, num1 As Double, num2 As Double)
    Select Case oprsign
        Case "+"
            arith = num1 + num2
        Case "-"
            arith = num1 - num2
        Case "*"
            arith = num1 * num2
        Case "/"
            arith = num1 / num2
        Case "%"
            arith = num1 Mod num2
        Case "^"
            arith = num1 ^ num2
        Case Else
            arith = "0"
    End Select
End Function

```

```
Public Function backing(otext As Object)
    If Len(otext) = 0 Or otext.Text = "0" Then
        backing = 0
    Else
        backing = Mid(otext.Text, 1, Len(otext) - 1)
    End If
End Function

Public Function mult(n As Integer, m As Integer) As Double
    If m = 0 Then
        mult = 0
    Else
        mult = mult(n, m - 1) + n
    End If
End Function
```

```

Public Function lcm(n As Long, m As Long, A() As Long)
    Dim i As Integer, p As Long
    p = 1
    i = 1
    While i <= UBound(A) - 1
        If n Mod A(i) = 0 Then
            n = n \ A(i)
            If m Mod A(i) = 0 Then
                m = m \ A(i)
            End If
            p = p * A(i)
        ElseIf m Mod A(i) = 0 Then
            m = m \ A(i)
        End If
        i = i + 1
    End While
    Return p * n * m
End Function

```

```

        If n Mod A(i) = 0 Then
            n = n \ A(i)
        End If
        p = p * A(i)
    Else
        i = i + 1
    End If
Wend
lcm = p
End Function

```

```

Public Function gcd(n As Long, m As Long)
    Dim p As Long, k As Long
    If n < m Then
        p = m
    Else
        p = n
    End If
    For k = 1 To p
        If n Mod k = 0 And m Mod k = 0 Then
            gcd = k
        End If
    Next k
End Function

```

```

Public Function division(n As Integer, m As Integer) As Double
    If n < m Then
        division = 0
    Else
        division = division(n - m, m) + 1
    End If
End Function

```

```

Public Function primenumber(lrange As Long, A() As Long)
    Dim j As Integer, i As Integer, p As Long, c1 As Long, k As Integer
    c1 = 0
    While j <= lrange
        For i = 2 To lrange
            If j Mod i = 0 And j <> i Then
                p = 0
                Exit For
            Else
                p = j
            End If
        Next i
        If p <> 0 Then
            c1 = c1 + 1
        End If
        j = j + 1
    Wend

```

```
ReDim A(c1 - 1)
```

```
j = 0
```

```
p = 0
```

```
While j <= lrange
```

```
For i = 2 To lrange
```

```
    If j Mod i = 0 And j <> i Then
```

```
        p = 0
```

```
        Exit For
```

```
    Else
```

```
        p = j
```

```
    End If
```

```
Next i
```

```
    If p <> 0 Then
```

```
        A(k) = p
```

```
        k = k + 1
```

```
    End If
```

```
    j = j + 1
```

```
Wend
```

```
End Function
```

The following three functions can be used to implement select sort algorithm

```
Public Function smaller(A() As Long, ni As Integer)
```

```
    Dim nk As Integer, lk As Long
```

```
    Dim nj As Integer
```

```
    lk = A(ni)
```

```
    nk = ni
```

```
    For nj = ni To UBound(A) '- length of the Array.
```

```
        If nj > ni Then
```

```
            If A(nj) <= lk Then
```

```
                lk = A(nj)
```

```
                nk = nj
```

```
            End If
```

```
        End If
```

```
    Next nj
```

```
    smaller = nk
```

```
End Function
```

```
Public Function swapping(A() As Long, ni As Integer, nj As Integer)
```

```
    Dim lk As Long
```

```
    lk = A(ni)
```

```
    A(ni) = A(nj)
```

```
    A(nj) = lk
```

```
End Function
```

```

Function selectsort(A() As Long)
Dim nj As Integer
  For nj = LBound(A) To UBound(A)
    swapping A, nj, smaller(A, nj)
  Next nj
End Function

```

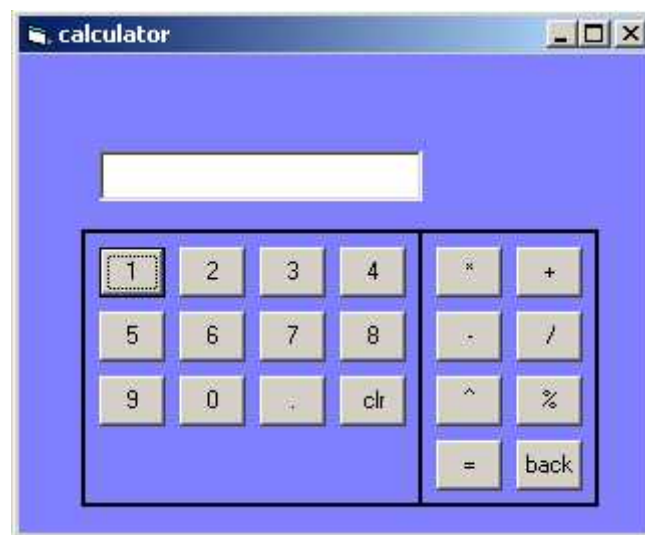
The function below implements insertion sort algorithm

```

Function insertion(A() As Long)
Dim nj As Integer, ni As Integer
Dim key As Long
  For nj = 2 To UBound(A)
    key = A(nj)
    ni = nj - 1
    While ni <> 0 And A(ni) > key
      A(ni + 1) = A(ni)
      ni = ni - 1
    Wend
    A(ni + 1) = key
  Next nj
End Function

```

IMPLEMENTATION OF A SIMPLE CALCULATOR MAKING USE OF THE ABOVE FUNCTIONS



Create the interface above then Type the code below in the code editor of the above interface to complete the calculator {Adjust the code appropriately to suit your interface}

```

Dim num1 As Double, num2 As Double, i As Integer
Dim oprflag As Boolean
Dim oprsign As String

```

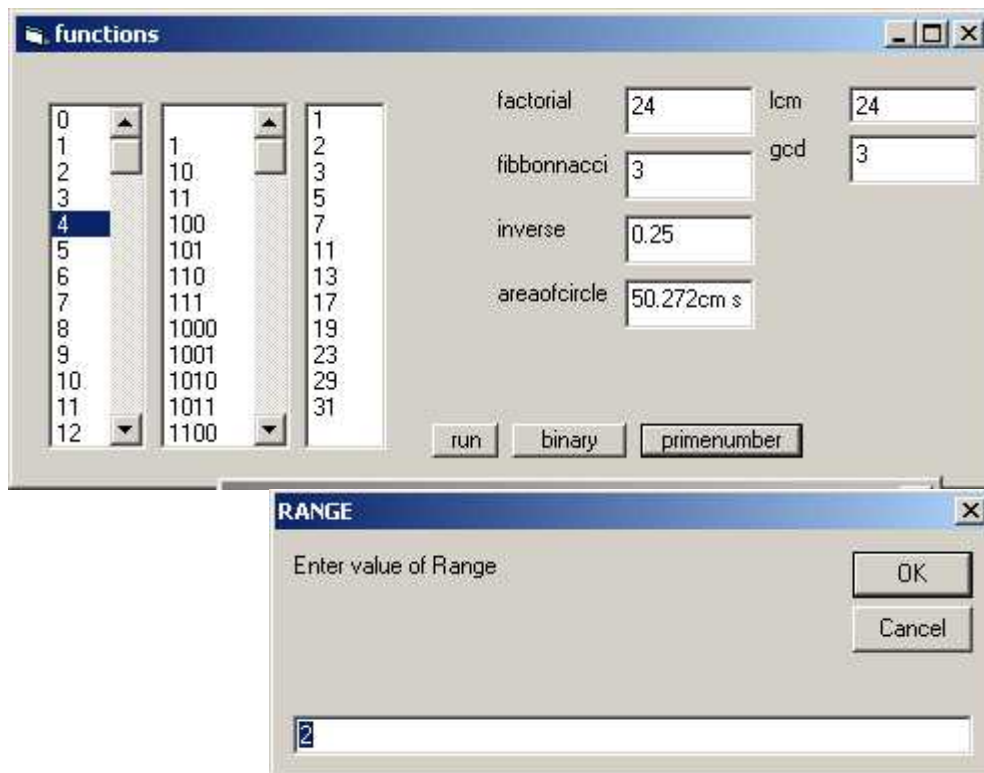
```

Private Sub Command1_Click(Index As Integer)
' assigning values to the text box by click individual
'command boxes
For i = 0 To 10
If Command1(i).Value = True Then
If oprflag = True Then
Text1.Text = Command1(i).Caption
oprflag = False
Else
Text1.Text = Text1.Text & Command1(i).Caption
End If
End If
Next i
'Clearing the text box
If Command1(11).Value = True Then
Call clearing(frmcalculator)
oprsign = ""
num1 = 0
num2 = 0
End If

'math operations
For i = 12 To 17
If Command1(i).Value = True Then
oprsign = Command1(i).Caption
num1 = Val(Text1.Text)
oprflag = True
End If
Next i
'results
If Command1(18).Value = True Then
num2 = Val(Text1.Text)
Text1.Text = arith(oprsign, num1, num2)
End If
'back cancellation
If Command1(19).Value = True Then
Text1.Text = backing(Text1)
End If
End Sub
Private Sub Form_Load()
oprflag = False
fillbuttons Command1
disable frmcalculator
clearing frmcalculator
End Sub

```

CREATE INTERFACE TO SHOW USE OF THE ABOVE FUNCTIONS WITHIN THE FORM



type the following code in the code view of the above interface to complete the implementation

{adjust the code to fit your application}

```
Dim arraypn() As Long
```

```
Dim arraypn1() As Long
```

```
Private Sub Command1_Click()
```

```
    Dim k As Integer
```

```
    While k < 100
```

```
        List1.AddItem k
```

```
        k = k + 1
```

```
    Wend
```

```
End Sub
```

```
Private Sub Command2_Click()
```

```
    Dim l As Integer
```

```
    Do While l < List1.ListCount
```

```
        List2.AddItem binary(CLng(List1.List(l)))
```

```
        l = l + 1
```

```
    Loop
```

```
End Sub
```

```
Private Sub Command3_Click()
```

```
    primenumber Val(InputBox("Enter value of Range", "RANGE", "2")), arraypn
```



```

For t% = LBound(arraypn) To UBound(arraypn)
    List3.AddItem arraypn(t)
Next t

```

```
End Sub
```

```
Private Sub Form_Load()
    cleartextbox Form6

```

```
End Sub
```

```
Private Sub List1_Click()
    Text1(0).Text = fact(List1.List(List1.ListIndex))
    Text1(1).Text = fibb(List1.List(List1.ListIndex))
    Text1(2).Text = inverse(List1.List(List1.ListIndex))
    nft = CInt(List1.List(List1.ListIndex))

```

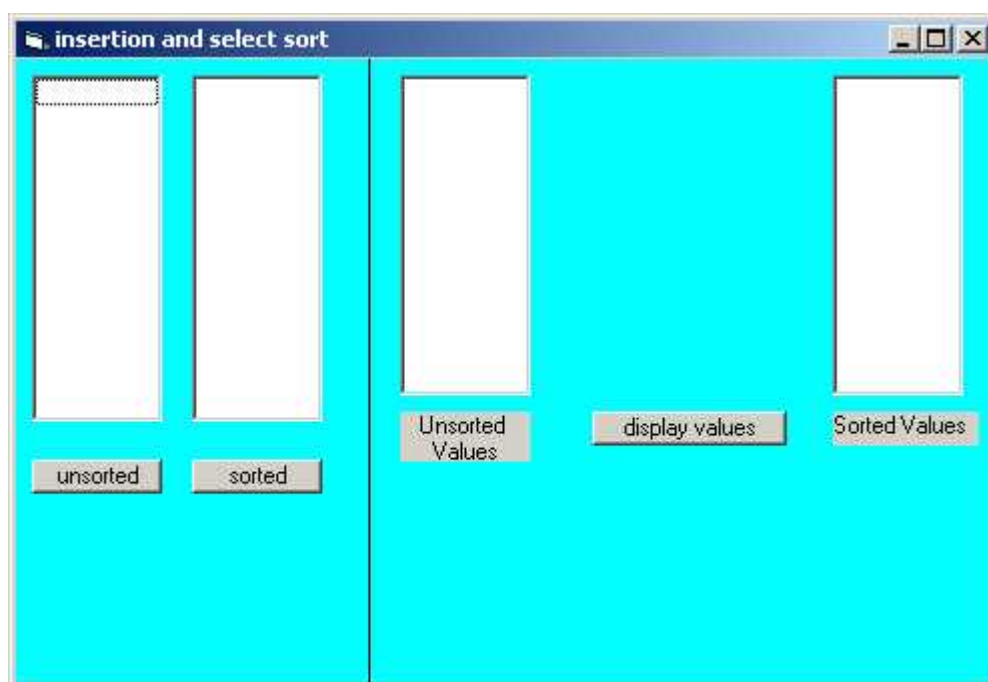
```
Text1(3).Text = pie * (nft ^ 2) & "cm square"
```

```
Text2.Text = lcm(CLng(Text1(0).Text), CLng(Text1(1).Text), arraypn)
Text3.Text = gcd(CLng(Text1(0).Text), CLng(Text1(1).Text))
End Sub
```

SHOWING THE OPERATION OF INSERTION AND SELECT SORT FUNCTIONS CREATED IN THE MODULES

Create the interface below then type the following code for the interface to be operational.

Adjust the code appropriately to fit your interface design



```
Dim Data(200) As Long, data1(20) As Long
Dim nj As Integer
Dim nm As Integer
```

```
Private Sub cmddisplay_Click()
```

```
    nj = 0
    List3.Clear
```

```
    data1(nm) = CLng(InputBox("Enter Numbers", "INSERTION SORT"))
    insertion data1
    While nj <= UBound(data1)
        List3.AddItem data1(nj)
        nj = nj + 1
    Wend
    nm = nm + 1
End Sub
```

```
Private Sub cmdsorted_Click()
```

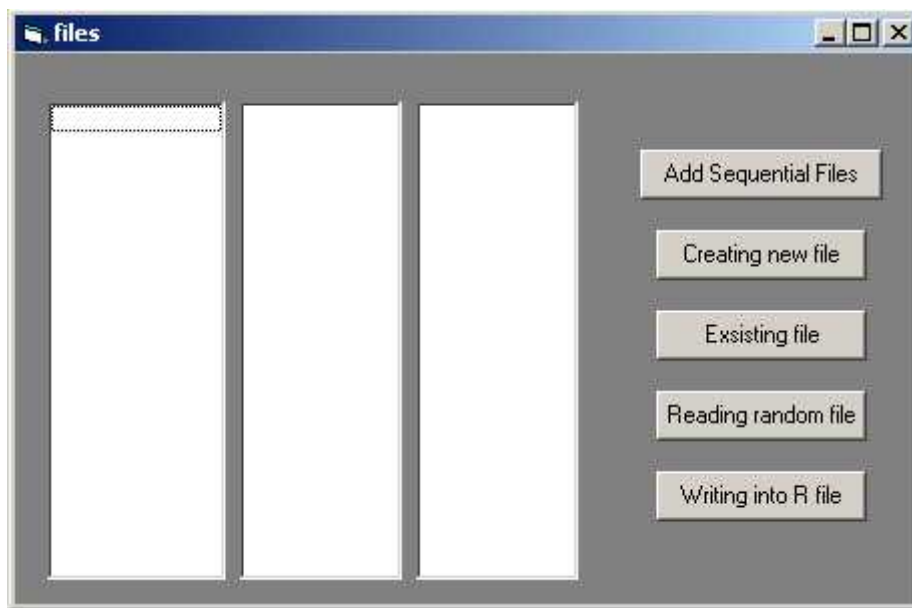
```
    selectsort Data
    nj = 0
    While nj <= 200
        List2.AddItem Data(nj)
        Debug.Print Data(nj)
        nj = nj + 1
    Wend
End Sub
```

```
Private Sub cmdunsorted_Click()
```

```
    nj = 0
    While nj <= 200
        Data(nj) = CLng(Rnd * 200)
        List1.AddItem Data(nj)
        nj = nj + 1
    Wend
End Sub
```

WRITING AND READING FROM RANDOM AND SEQUENTIAL FILES

Create the interface below that will be used for implementation of file access



Write the code below in the interface code view to complete the implementation{Adjust the code appropriately to suit your self created interface}

```
Dim Data(200) As Long, data1(20) As Long
Dim nj As Integer
Dim nm As Integer
```

```
Private Sub cmddisplay_Click()
```

```
    nj = 0
    List3.Clear
```

```
    data1(nm) = CLng(InputBox("Enter Numbers", "INSERTION SORT"))
    insertion data1
    While nj <= UBound(data1)
        List3.AddItem data1(nj)
        nj = nj + 1
    Wend
    nm = nm + 1
End Sub
```

```
Private Sub cmdsorted_Click()
    selectsort Data
    nj = 0
    While nj <= 200
        List2.AddItem Data(nj)
        Debug.Print Data(nj)
        nj = nj + 1
    Wend
```

End Sub

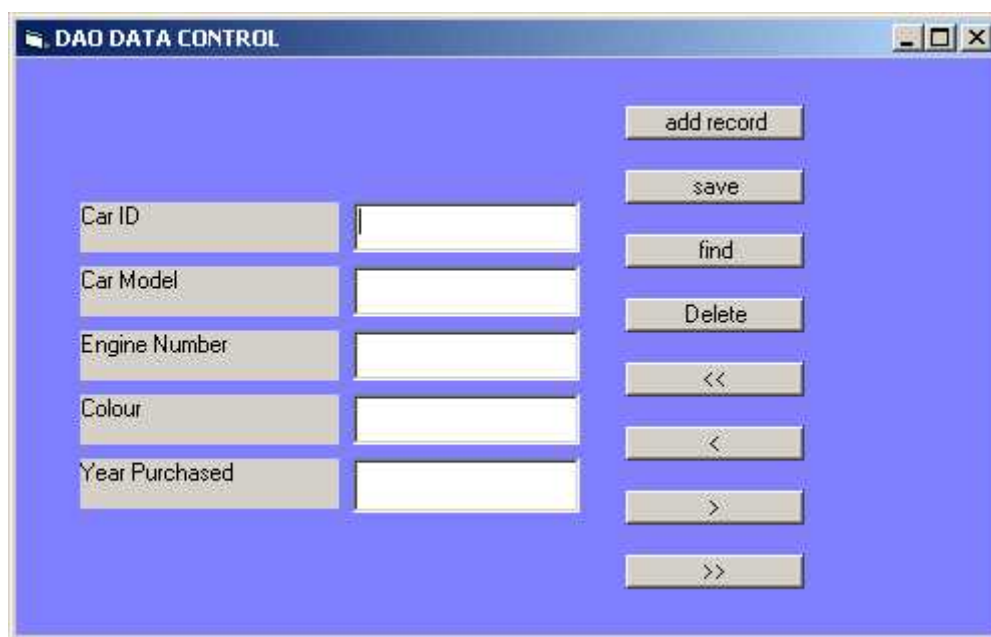
```
Private Sub cmdunsorted_Click()
    nj = 0
    While nj <= 200
        Data(nj) = CLng(Rnd * 200)
        List1.AddItem Data(nj)
        nj = nj + 1
    Wend
```

End Sub

DATA BASE ACCESS USING THE DATA CONTROL OBJECTS BOTH COMMAND DRIVEN AND GRAPHICAL

Create the data base db4.mdb and db5.mdb in the same folder as the application and created the tables applicable with the fields.

Assignment 1.0(Data Access Object)



Give the labels names equivalent to there functionality, e.g. for command button addrecord the name will be <cmdaddrecord> ,for label car id the names will be lblcarid and for first textbox adjacent to label lblcarid will be txtcarid

After implementation of the above design you can type the code below adjusting the code to fit into the procedures according to how you have captioned or named the design objects.

Select from project menu-reference menu item dialogue one of the Microsoft DAO (2.5,2.5/3.51,3.51,3.60) object library

DAO connectivity code.

In the code view type the following code

DECLARATION

```
Dim db As Database 'declaration of the database variable
Dim rs As Recordset 'declaration of the recordset
```

'Clearing the textboxes to allow data entry

```
Private Sub cmdadd_Click()
clearing 'calling a procedure to clear the textbox
End Sub
```

'Deleting the current record

```
Private Sub cmddelete_Click()
'deleting records

If Text1.Text <> "" Then
    If MsgBox("Are you sure that you want to delete this record", vbInformation +
vbYesNo, "DELETING") = vbYes Then
        rs.Delete
        MsgBox "record has been deleted"
    Else
        MsgBox "RECORD WAS NOT DELETED", vbInformation + vbOKOnly,
"DELETING"
    End If
End If
End Sub
```

Searching for records already existing in the database

```
Private Sub cmdfind_Click()
'if using table record set

'rs.Index = "car id"
'rs.Seek "=", Val(InputBox("Enter Car ID", "SEARCHING"))
'If rs.NoMatch = False Then

' if using either snapshot or dynaset recordset

rs.FindFirst "[car id] =" & InputBox("Enter car id", "FINDING")
If rs.EOF = False Then
    getvalues
Else
    MsgBox "RECORD DOES NOT EXIST", vbInformation + vbOKOnly, "FINDING"
End If
End Sub
```

Saving the records into the database

```
Private Sub cmdsave_Click()
' committing data from the textboxes to the database
rs.AddNew
setvalues
```

```
rs.Update
End Sub
```

'Navigating data

```
Private Sub Command5_Click()
If Not rs.BOF Then
    rs.MoveFirst
End If
    getvalues
End Sub
```

```
Private Sub Command6_Click()
If Not rs.BOF Then
    rs.MovePrevious
End If
    getvalues
End Sub
```

```
Private Sub Command7_Click()
If Not rs.EOF Then
    rs.MoveNext
End If
    getvalues
```

```
End Sub
```

```
Private Sub Command8_Click()
If Not rs.EOF Then
    rs.MoveLast
End If
    getvalues
End Sub
```

Linking the current database

```
Private Sub Form_Load()
Set db = DBEngine.Workspaces(0).OpenDatabase(App.Path & "\db4.mdb")
'Set rs = db.OpenRecordset("car", dbOpenTable) 'table recordset
Set rs = db.OpenRecordset("select all* from car", dbOpenSnapshot)
clearing

End Sub
```

Procedure to retrieve data from the database

```
Private Sub getvalues()
If rs.BOF = False And rs.EOF = False Then

    Text1.Text = rs.Fields("Car ID")
    Text2.Text = rs.Fields("Car Model")
    Text3.Text = rs.Fields("Engine Number")
    Text4.Text = rs.Fields("Colour")
    Text5.Text = rs.Fields("Year Purchased")
```

```
End If
End Sub
```

Procedure to assign data to the relevant fields in the database

```
Private Sub setvalues()
    rs![car id] = Text1.Text
    rs![car model] = Text2.Text
    rs![engine number] = Text3.Text
    rs![colour] = Text4.Text
    rs![year purchased] = Text5.Text
End Sub
```

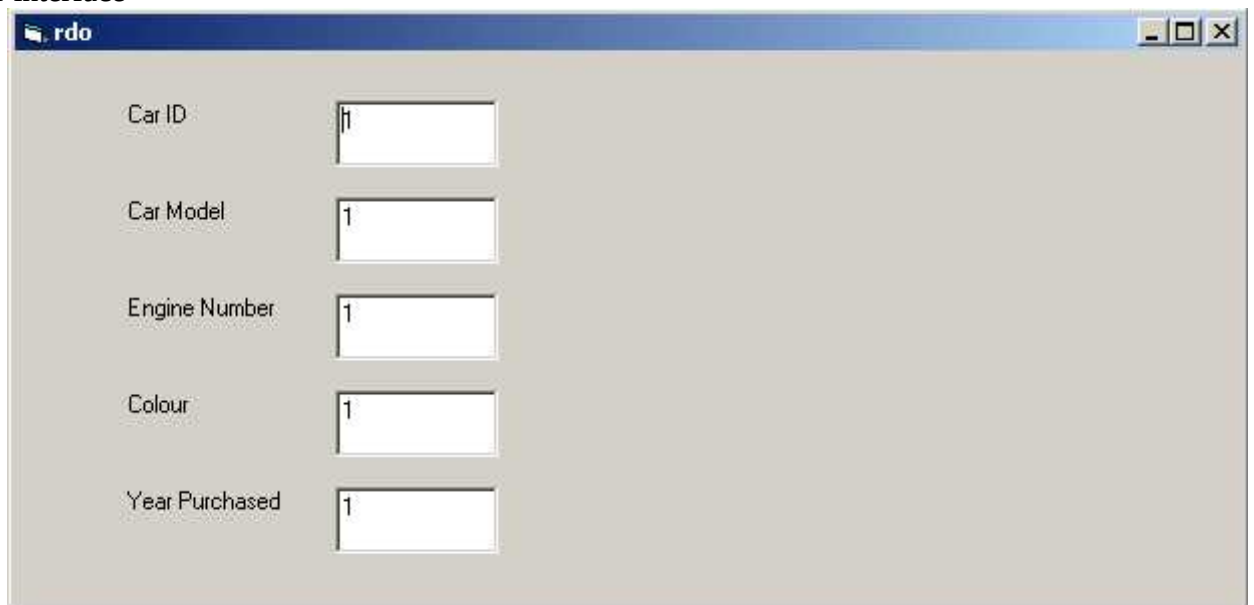
Procedure for clearing textboxes

```
Public Sub clearing()
    Dim myclearing As Control
    For Each myclearing In Me.Controls
        If TypeOf myclearing Is TextBox Then myclearing.Text = ""
    Next
End Sub
```

Assignment1.1 (rdo-Remote Data Object)

Select from project menu-reference menu item dialogue one of the Microsoft Remote Data Object (2.0) object library

Create the interface below then type the code that follows adjusting them appropriately to fit your interface



The screenshot shows a Windows application window titled 'rdo'. Inside the window, there are five text boxes arranged vertically. Each text box has a label to its left: 'Car ID', 'Car Model', 'Engine Number', 'Colour', and 'Year Purchased'. Each of these text boxes contains the number '1'. The window has a standard Windows title bar with minimize, maximize, and close buttons.

CODES

```
Dim rsdat As rdoConnection
Dim rdrecord As rdoResultset

Private Sub Form_Load()
```

```

DBEngine.RegisterDatabase "mydsn", "Microsoft Access Driver (*.mdb)", True,
App.Path & "\db4.mdb"
Set rddata = rdoEngine.rdoEnvironments(0).OpenConnection("mydsn")
Set rdrecord = rddata.OpenResultset("select*from Car", rdOpenKeyset)

getvalues
End Sub

```

```

Private Sub getvalues()
Text1.Text = rdrecord.rdoColumns(0).Value
Text2.Text = rdrecord.rdoColumns(0).Value
Text3.Text = rdrecord.rdoColumns(0).Value
Text4.Text = rdrecord.rdoColumns(0).Value
Text5.Text = rdrecord.rdoColumns(0).Value
End Sub

```

Assignment1.2 (adodb-Activex Data Object Data Base)

Select from project menu-reference menu item dialogue one of the Microsoft Activex Data Object (2.0,2.1,2.3,2.5,2.6,2.7) object library

Create the interface below then type the code that follows adjusting them appropriately to fit your interface

CODES

```

Dim cn As adodb.Connection
Dim rs As adodb.Recordset

```

```

Private Sub cmdadd_Click()
clearing
enable
End Sub

```

```

Private Sub cmddelete_Click()

```



```

If Text1.Text <> "" Then
    If MsgBox("Are you sure you want to delete therecord", vbCritical + vbYesNo,
"STUDENT") = vbYes Then
        rs.Delete (adAffectCurrent)
        MsgBox "The record has been deleted"

```

```

    Else
        MsgBox "The record was not deleted"
    End If
    rs.Requery
Else
    MsgBox "No record to delete"
End If
clearing
disable
End Sub

```

```

Private Sub cmdfind_Click()
    rs.Find "Name=" & InputBox("Enter student name", "STUDENT") & "", 0,
adSearchForward, 1
    If rs.EOF = False Then
        get_data
    Else
        MsgBox "The records do not exist"
    End If
End Sub

```

```

Private Sub cmdsave_Click()
    rs.AddNew
    set_data
    rs.Update
End Sub

```

```

Private Sub Form_Load()
    Set cn = New adodb.Connection
    cn.Open "provider=Microsoft.jet.oledb.4.0; data source=" & App.Path & "\db5.mdb"
    Set rs = New adodb.Recordset
    rs.Open "select * from admission", cn, adOpenDynamic, adLockOptimistic,
adCmdText
    disable
    clearing
    get_data
End Sub

```

```

Public Sub get_data()
    Text1.Text = rs!Name
    Text2.Text = rs![Registration Number]
    Text3.Text = rs![Date of Birth]
    Text4.Text = rs![Fee Payable]
End Sub

```

```

Public Sub set_data()
    rs!Name = Text1.Text
    rs![Registration Number] = Text2.Text

```

```
rs![Date of Birth] = Text3.Text
rs![Fee Payable] = Text3.Text
End Sub

Public Sub clearing()
    Dim myclear As Control

    For Each myclear In Me.Controls
        If TypeOf myclear Is TextBox Then
            myclear.Text = ""
        End If
    Next
End Sub

Public Sub enable()
    Dim myenable As Control

    For Each myenable In Me.Controls
        If TypeOf myenable Is TextBox Then
            myenable.Enabled = True
        End If
    Next
End Sub

Public Sub disable()
    Dim mydisable As Control

    For Each mydisable In Me.Controls
        If TypeOf mydisable Is TextBox Then
            mydisable.Enabled = False
        End If
    Next
End Sub

Private Sub Command3_Click()
    If Not rs.EOF Then
        rs.MovePrevious
    End If
    get_values
End Sub

Private Sub Command4_Click()
    If Not rs.EOF Then
        rs.MoveNext
    End If
    get_values
End Sub

Private Sub Command5_Click()
    If Not rs.EOF Then
        rs.MoveFirst
    End If
    get_values
End Sub
```

```

Private Sub Command6_Click()
    If Not rs.EOF Then
        rs.MoveLast
    End If
    get_values
End Sub

```

Assignment1.3 (activex data object data control)

Select from project menu-component menu item dialogue one Microsoft ADO Data Control 6.0 (sp6) OLEBD

Create the interface below and draw the adodc object on the form surface then right click the control object then follow on the instruction that follows to connect the interface to the database and then type the code that follows adjusting them appropriately to fit your interface

	Name	Registration Number	Date of Birth	Fee Payable
▶	Trevor Kimani	34566	12/04/1980	10000
	Julie Atieno	76543	01/01/1975	12000
	peter clay	74744	12/02/1979	13000

CODES

```

Private Sub cmdadd_Click()
    'using the addnew method to create copy buffer space to allow editing of
    'of data.
    'Text box one inassigned values using the input box
    'and focus is set to the next textbox after assigning the first textbox with items

```

```

    Adodc1.Recordset.AddNew
    txtName.Text = InputBox("Enter student name", "STUDENT")
    txtRegistration.SetFocus

```

```

End Sub

```

```

Private Sub cmdclose_Click()
    'remove the form from memory and display
    Unload Me
End Sub

```

```
Private Sub cmddelete_Click()  
    If MsgBox("Are you sure you want to delete therecord", vbCritical + vbYesNo, "STUDENT")  
= vbYes Then  
        Adodc1.Recordset.Delete (adAffectCurrent)  
        MsgBox "The record has been deleted"  
  
    Else  
        MsgBox "The record was not deleted"  
    End If  
    Adodc1.Recordset.Requery  
  
End Sub  
  
Private Sub cmdfind_Click()  
    'used to find a specific record in the table  
    Adodc1.Recordset.Find "Name=" & InputBox("Enter student name", "STUDENT") & "", 0,  
adSearchForward, 1  
End Sub  
  
Private Sub cmdsave_Click()  
    'commit the data to the database  
    If txtName.Text <> "" Then  
        Adodc1.Recordset.Update  
        MsgBox "The record has been saved", vbSystemModal + vbInformation, "STUDENT"  
    Else  
        MsgBox "The record has not been saved"  
    End If  
End Sub
```